

1 Att programmera i C

Programmeringsspråket C är ett av de vanligaste språken att programmera på låg nivå. Det används för operativsystem, drivrutiner, inbäddade system och mycket mer. Några exempel är Linux (kärnan), ffmpeg (bibliotek för kodning/avkodning av media) och gkt+ (widget biblioteket).

C är ofta ett bra val för maskinnära program då språket självt både är relativt maskinnära och definierar en profil utan stödbibliotek eller operativsystem. C anses ofta osäkert och krångligt, detta har dock inte hindrat att många framgångsrika program är skrivna i det och devisen "Trust the Programmer" som på många sätt genomsyrar språket har antagligen gjort sin del för göra det populärt.

2 C är inte C++

En vanlig missupfattning som är värd att klara upp på en gång är att C skulle vara samma sak som C++. Det är det inte. Alls. Överhuvudtaget.

C++ är ett programmeringsspråk som är konstruerat för att vara likt C, men skillnaderna mellan språken är tillräckligt stora att det i allmänhet inte är möjligt att kompilera ett C program med en C++ kompilator eller tvärtom. Visserligen finns det program i C som kan kompileras med en C++ kompilator, men å andra sidan finns det program vars källkod är både COBOL, Pascal, Fortran, C, PostScript, sh-script, maskinkod och Perl samtidigt¹ så det betyder inte så mycket som man kan tro.

3 Maskinnära C

Anledningen till att C ofta betraktas som maskinnära är att de olika konstruktioner i form av loopar, funktioner, strukturer o.dyl. som återfinns återspeglar många vanliga konstruktioner som används när man programmerar assembler. I motsats till exempelvis språk som C++ är C mycket mer explicit och ur kompileringssynpunkt är det snarast ett WYSIWYG språk.

Med lite träning kan man ofta direkt översätta C till assembler manuellt men här, liksom med allt annat maskinnära måste man vara noggran med detaljerna. Ett exempel på ett vanligt fel vid sådana övningar är att man placerar hoppvillkoret i en loop fel².

3.1 Pekare

Någonting som visserligen inte är helt unikt för C, men som det sällan talas högt om (än mindre uppmuntras) i andra språk är direkt minneshantering med pekare. C har en otypad minnesmodell där ett minnesblock helt enkelt är en sekvens av bytes — vad man gör med minnet är upp till användaren.

Vad detta innebär i praktiken är att fysiska datastrukturer är relativt enkla att manipulera i C då deras faktiska minnesrepresentation är direkt tillgänglig. Givetvis medför detta risker också och C, liksom assembler, ställer högre krav på den som programmerar.

¹se <http://ideology.com.au/polyglot>

²Vidare utredning av detta lämnas som övning åt läsaren.

3.1.1 Arrayer

En vanlig missupfattning om C är att pekare och arrayer är samma sak. I C kan man deklarera arrayer, men normalt kan man inte använda en array. Anledningen till detta är att en arrayvariabel vid användning ger en pekare (OBS! *ger* en pekare, *inte* "är en pekare") till arrayen, som är ett minnesblock med en sekvens av bastypen. (en array som deklareras "`int a[4]`" resulterar alltså i en variabel som ger en pekare till $4 * \text{sizeof}(\text{int})$ bytes).

En array är inte ett giltigt *lvalue*, d.v.s. en arrayvariabel kan inte tilldelas ett värde utan det är endast elementen i arrayen som kan ändras. Jämför detta med en pekarvariabel som kan ändras att peka på olika minnesområden.

3.2 Bitoperatorer

En typ av operationer som ofta är användbara när man programmerar mot hårdvara (eller med bilddata för den delen) är operationer på bitnivå. Dessa påverkar på olika sätt bitarna i en variabel/ett värde och för att använda dem effektivt bör man förstå hur binära tal fungerar.

3.2.1 `&` — `and`

Bitvis and mellan två tal. Används ofta för maskning och bit-test. Exempel:

```
if(value&2) {  
    printf("Bit #1 är satt.\n");  
}
```

3.2.2 `|` — `or`

Bitvis or mellan två tal. Används ofta för att maska in värden.

3.2.3 `^` — `xor`

Bitvis xor (exklusivt eller) mellan två tal. Används ofta för att växla bitar. Exempel:

```
v^=8;    /* Växla bit 3 */
```

3.2.4 `~` — `not`

Invertering av alla bitar i ett värde. Används ofta tillsammans med maskning för att konstruera ett värde. Exempel:

```
if((x|y)&(~511)) {  
    printf("Minst en av x och y är större än 511\n");  
}
```

3.2.5 `<<` — `shift left`

Vänsterskift av värde. Vänsterskift n steg har samma effekt som multiplikation med 2^n förutom att overflow inte är applicerbart på skift.

3.2.6 >> — shift right

Högerskift av värde. I teorin är det fritt fram för kompilatorn att välja aritmetiskt (med tecken) och logiskt (utan tecken) skift hur den vill, men i praktiken utförs logiskt skift på “unsigned” värden och aritmetiskt på “signed” värden. Exempel:

```
signed int a;
...
a=1+(a>>1); /* aritmetiskt skift, a=1+a/2 */
```

Det bör även noteras att skift kan användas för teckenförläggning i C.

4 Kompilering

Ett program skrivet i C är, precis som alla andra program som inte är skrivna i maskinkod, inte direkt körbart. För att omvandla källkoden till ett körbart program används en kompilator och resultatet är ett program i maskinkod. Transformationen sker dock inte i ett steg, utan är en process med ett antal mellansteg vilka vi nu skall se närmre på.

4.1 Preprocessorn

Det första steget, vilket även förekommer i C++ även om det sällan nämns, är pre-processorn. Preprocessorn hanterar i huvudsak två olika saker, det första är inkludering av andra filer och det andra är definition av makron. Om vi tittar på följande exempel:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    printf("Hello World\n");
    return(EXIT_SUCCESS);
}
```

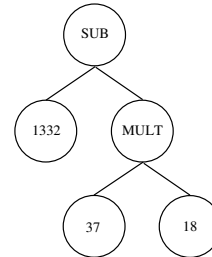
Detta korta program innehåller två direktiv för att inkludera andra filer och använder ett makro. Direktivet **#include** anger att en fil skall kopieras in i programmet och i filen “**stdlib.h**” definieras **EXIT_SUCCESS** som preprocessorn sedan ersätter med ett värde. Definitionen kan exempelvis se ut såhär:

```
#define EXIT_SUCCESS (0L)
```

Vilket betyder att “**EXIT_SUCCESS**” skall ersättas med “(0L)”. Makron kan vara betydligt mer komplexa än så och kan ta parametrar, men för enkelhetens skull nöjer vi oss här med att konstatera att så är fallet.

4.2 Abstrakt representation

När programtexten är redo att behandlas omvandlas den först till en abstrakt representation, ofta kallat AST eller Abstract Syntax Tree. Denna representation kanske inte ens lagras någonstans men är ett resultat av att kompilatorn läser källkoden. Anledningen till att man kallar detta ett träd (eller parse-träd) är att ett träd innehåller all information som behövs — vi kan som exempel se på hur uttrycket “1332-37*18” representeras.



4.3 Mellankod

Många (men inte alla) kompilatorer har en intern pseudo-assembler representation av koden. Anledningen till och användningen av detta ligger dock utanför denna kursens omfång.

4.4 Assembler och objektkod

Generering av objektkod, d.v.s körbar kod, kan ske på lite olika sätt. Tidigare var det vanligt att en kompilator genererade assemblerkod som sedan skickades till en assembler, men nu är det vanligare att objektkoden genereras direkt från den interna representationen.

I fallet med assembler — och även assembler genererad av en kompilator — har assemblatorn ett par uppgifter som underlättar kodskrivning. Eftersom vi redan sett på makron behandlar vi inte detta vidare här, utan tittar endast på adressgenerering. En av de kanske mindre uppenbara skillnaderna mellan maskinkod och assembler är att man i assembler kan använda namn på minnesadresser — skriver man maskinkod måste man ange adresserna direkt. Detta medför att assemblatorn vid assemblering måste omvandla namnen till adresser, vare sig relativa eller absoluta. Om vi tar följande exempel i MIPS assembler:

```
.data
dummy:
    .word    17
datalen:
    .word    0x10
    .text
main:
    la      $t0,datalen
```

Detta översätts först genom makro-expansion (*la* är en syntetisk instruktion) och sedan byts “datalen” ut mot motsvarande numeriska adress och slutresultatet kan (i assembler) se ut ungefär såhär:

```

        .data
dummy:
        .word    17
datalen:
        .word    0x10
        .text
main:
        lui      $t0,0x0100
        ori      $t0,$t0,0x0004

```

Notera att namnet “datalen” bytts ut mot adressen 0x01000004. Hur kommer det sig då att assemblern kan veta vilken adress som datalen har? Det enda assemblern behöver veta är att programmets datasektion börjar på adress 0x01000000, varpå det är en enkel sak att räkna ut adressen för olika data.

Nu är det visserligen inte nödvändigtvis så att datasektionen alltid hamnar på den adressen (eller ens samma adress i två olika körningar av samma program), men detta är upp till operativsystemet att se till att programmet får veta — antingen genom att datasektionen för alla program hamnar på samma adress, genom att basadressen för datasektionen står i den körbara filen eller genom att operativsystemet vid programstart skickar med basadressen.

Det är viktigt att notera att det är inte någonting “magiskt” med att generera körbara program. Alla programmeringsspråk som kan skriva binära filer kan i princip även skriva körbara program direkt — även om den del språk kanske är lättare att använda för ändamålet än andra.